

Business Entity Resolution: our approach

Amazon ML Challenge 2026, Round 1. Written for the team before we split the work.

1. What the task actually is

We get business records from three sources. The same shop can appear in all three, spelled differently each time, with no shared ID linking them. Source 1 is the clean master list. For every Source 1 record, we have to output every Source 2 and Source 3 record that is the same business. It can be none, one, or several. The problem statement says the average is closer to several.

2. How the score works, and why it changes our strategy

The metric is F0.5, averaged separately for each Source 1 record. The formula is given in the problem statement. The important part is that precision counts twice as much as recall, so a wrong match hurts us more than a missed one.

Worked through on one record, that means:

- **Right match, nothing extra:** we get the full 1.0
- **Right match plus one wrong one:** we drop to about 0.56
- **Correct match missed entirely:** we get 0.0
- **Any guess on a record that truly has no match:** we lose the whole point for that record

So when the model is unsure, staying quiet is usually the better bet. That one idea drives most of the design below.

3. Four things we measured in the data

These are not assumptions. They come from our own profiling script (`src/profile_data.py`), run over the full training set.

- **Singletons are rare.** Only 5.58 percent of Source 1 records have no match at all. Submitting nothing for everyone would score 0.0558, so that is our floor, not a plan.
- **Each record has at most one owner.** No Source 2 or Source 3 record is ever claimed by two different Source 1 records. We checked all 7,638,365 links and found zero exceptions. We can therefore force our predictions to be exclusive, which costs a little recall and buys precision. Precision is what the metric pays for.
- **Matches never cross countries.** Across 1,038,755 sampled links, not one match crossed a country label. That makes country a free filter that cuts the search space by roughly 2.6 times with no loss at all.
- **Roughly one in ten Indian matches is in a different script.** Source 1 is always in Latin script, but about 10 percent of the Indian matches point at a Devanagari record. Character and word overlap score exactly zero across two different scripts, so those matches are invisible unless we convert first. We wrote our own converter, since the rules forbid outside services.

4. The pipeline, five steps

The shape of this is standard entity resolution, and the problem statement points at it directly in its own Tips for Success section, which tells us to invest in blocking and to look at Jaccard, Levenshtein and TF-IDF cosine.

Step 1. Clean the text. Lowercase, drop punctuation, strip accents so the French records line up, convert Devanagari to Latin, and fold abbreviations so Pvt Ltd and Private Limited become the same thing. Street numbers and PIN codes get pulled into their own field, since they separate businesses far better than words like Road do.

Step 2. Blocking, also called candidate generation. We cannot compare 1.7 million records against 10 million, that is 17 trillion pairs. So for each record we shortlist a handful of plausible partners using TF-IDF cosine on the name, within the same country. This step decides the highest recall we can ever reach, because anything it misses is gone for good.

Amazon asks us to submit this candidate list as a separate file and reviews its quality, so it is graded work, not scratch work.

Step 3. Build comparison features. For each shortlisted pair, measure how alike the two records are: name similarity, address similarity, how many rare words they share, whether the house number and PIN code match. Sharing an unusual word like Zephay tells us much more than sharing the word Restaurant, so rare words get weighted higher.

Step 4. Train a classifier. Feed those features to a gradient boosting model, LightGBM. It learns which combinations mean same business. The negative examples have to be the near misses our own blocking produced, not random pairs, because random pairs are too easy and teach it nothing.

Step 5. Pick the cut off, then enforce exclusivity. Sweep the accept threshold against F0.5 on our own held out data and take the best one. Expect it to sit high, because the metric punishes false matches. Then apply finding number two: if two Source 1 records both claim the same partner, only the stronger claim survives.

5. Why we are building it in this order

Steps 1 and 2 block everything else, so they go first and they cannot slip. A weak shortlist caps the final score no matter how good the model is. We also want a simple rules only version submitted on day one, before any machine learning, because it proves our output format is accepted and gives us a real leaderboard number to compare against. Getting a format error on the last night is the failure mode we are designing around.

6. One trap we are planning for

The training data only covers the US and India. The test set adds France, which we have never seen. If anything in our code quietly learns US or Indian address habits, we lose 15 percent of the final score. Our check is cheap: train on US records only, test on Indian records only. If that collapses, we have a problem worth fixing before day three.

7. What to pick up

Claim a phase in the group chat before you start so we do not build the same thing twice. Full detail for every phase is in the README on GitHub.

- Steps 1 and 2 are the bottleneck. Two people on blocking is not wasted effort.
- Log every run in experiments/experiments.md, including the ones that fail. If it is not logged we cannot compare it or write it up later.
- Nothing gets uploaded unless it beats our best local score and the official validator passes. We only get five uploads a day.

Everything above is built from the provided training data only. No outside databases, no lookup services, no geocoding APIs. That is a disqualification rule, and it is worth all of us knowing it.